

```
@page { margin: 14mm 12mm; } body { font-family: "DejaVu Sans",  
Arial, sans-serif; font-size: 10.5pt; line-height: 1.34; color: #1a2733;  
max-width: 100%; margin: 0; } h1 { font-size: 19pt; color: #12324b;  
border-bottom: 2px solid #12324b; padding-bottom: 3px; } h2 { font-  
size: 14pt; color: #12324b; border-left: 4px solid #12324b; padding-  
left: 7px; margin-top: 16px; } h3 { font-size: 12pt; color: #234;  
margin-top: 12px; } table { border-collapse: collapse; width: 100%;  
margin: 8px 0; font-size: 9pt; } th, td { border: 1px solid #b7c2cc;  
padding: 3px 6px; text-align: left; vertical-align: top; } th {  
background: #12324b; color: #fff; } tr:nth-child(even) td {  
background: #f2f5f7; } code { background: #eef1f4; padding: 1px  
3px; border-radius: 3px; font-size: 8.8pt; } pre { background:  
#0f1b26; color: #dfe7ee; padding: 8px 10px; border-radius: 5px; font-  
size: 8.4pt; overflow-x: auto; white-space: pre-wrap; } pre code {  
background: transparent; color: inherit; }
```

FUNCTIONAL DESIGN

O-RAN VIP Suite — Functional Design (FD) Document

Author: AVIK MAJUMDAR **Project:** AVIK VIP FACTORY — O-RAN VIP Suite **Document:** Functional Design (FD) **Revision:** 1.0 **Date:** 2026-08-09

1. Introduction

This Functional Design document describes the functional intent, field-level model, legal space, cross-field constraints, coverage model, and assertion set of every component in the O-RAN VIP Suite. It is the design counterpart to the Verification Reference Document: the VRD says *how* the suite verifies; the FD says *what* is being verified. Requirements are DERIVED from eCPRI v2.0, O-RAN.WG4.CUS/MP, IEEE 1914.3/1588v2, and 3GPP TS 38.211 / 36.211.

2. Common Design Model

Every component is modelled at the transaction (header) level as a **64-bit packed protocol header**. The verification datapath is a combinational codec:

- `pack(fields)` → 64-bit wire vector — explicit bit-field concatenation, MSB-first.
- `unpack(wire)` → `fields` — the exact inverse slicing.
- `is_legal(fields)` → bit — conjunction of per-field legality and cross-field constraints.
- `first_violation(fields)` → string — returns the name of the first failing check (used to name the fired check in negative tests), or empty string if legal.

Correctness is defined as: (a) round-trip identity `unpack(pack(x)) == x`; (b) agreement of the DUT packer with an independent golden packer produced by the Z3/Python generator; and (c) legality of every generated transaction. The 64-bit width is a deliberate design choice giving a uniform golden word and clean single-word round-trip across all thirteen components; fields are padded with a reserved field to exactly 64 bits.

2.1 Legal-space primitives

Primitive	Meaning	Example
const v	field fixed to a constant	version == 1
enum {...}	field $\in$ discrete legal set	msg_type $\in$ {0..7}; num_ports $\in$ {1,2,4,8,16,32,64}
range [lo:hi]	field $\in$ inclusive range	payload_size $\in$ [8:1024]
free	full width legal	pc_id (16-bit, any)
cross	relation between fields	start_prb + num_prb $\leq$ 275

2.2 Coverage primitives

enum $\rightarrow$ one bin per legal value; bool $\rightarrow$ {0,1}; wrap $\rightarrow$ {min, max, mid} (boundary + interior); bucket4 $\rightarrow$ four equal range buckets. A cross coverpoint pairs the two most significant coverpoints. Native covergroups (Lane-2B) add illegal_bins that must stay empty under legal stimulus.

3. Per-Component Functional Design

Each subsection gives the component purpose, the field model, the legal space, cross constraints, the coverage model, and the assertion set.

3.1 ecpri_transport — eCPRI Transport Layer (both modes)

Purpose: model the eCPRI v2.0 common header (§3.1) and the Type-0 IQ payload header (§3.2) used by all 5G planes and 4G-over-eCPRI. Revision-stable across CUS releases.

Field	Width	Legal	Cover
version	4	const 0x1	—
rsvd	3	const 0	—
concat	1	{0,1}	bool
msg_type	8	[0:7]	enum
payload_size	16	[8:1024]	bucket4
pc_id	16	free	—
seq_id	8	[0:255]	wrap
e_bit	1	{0,1}	bool
sub_seq	7	[0:127]	—

Assertions: recovered version == 0x1; msg_type $\leq$ 7; payload within [8:1024]; round-trip identity; sequence-number continuity (Lane-2B concurrent SVA).

3.2 cpri_eth — CPRI-over-Ethernet (4G LTE)

Purpose: 4G baseline — IQ data mapping over Ethernet, six bandwidth profiles, SyncE, and the C&M slow plane.

Field	Width	Legal	Cover
version	4	const 0x1	—
bw_profile	4	{0..5} = 1.4/3/5/10/15/20 MHz	enum
direction	1	{0,1} DL/UL	bool
iq_rate_id	4	[0:15]	—
sync_eql	4	{0,2,4,11,15} QL codes	enum

frame_id	10	[0:1023]	—
subframe	4	[0:9]	—
cm_flag	1	{0,1}	bool
seq	8	[0:255]	wrap

Coverage: all 6 BW profiles × direction; SyncE QL states; sequence wrap. Assertions: frame timing period; BW-profile IQ-rate consistency; C&M/IQ separation.

3.3 uplane — U-Plane Split 7.2x (5G NR)

Purpose: user-plane IQ delivery — Section Types 1/3/5/6, PRB allocation, numerology, symbol map, and IQ compression selection.

Field	Width	Legal	Cover
version	4	const 0x1	—
section_type	4	{1,3,5,6}	enum
numerology	4	[0:4] (μ)	enum
start_prb	9	[0:273]	—
num_prb	9	[1:275]	bucket4
symbol_id	4	[0:13]	—
comp_type	2	{0,1,2,3} = none/BFP/ μ -law/static	enum
comp_bits	5	[9:16]	—
seq	8	[0:255]	wrap

Cross: $\text{start_prb} + \text{num_prb} \leq 275$ (FR1 PRB ceiling). Assertions: PRB sum bound; symbol in [0:13]; compression type valid; round-trip.

3.4 cplane — C-Plane Split 7.2x (5G NR)

Purpose: control-plane section scheduling — Section Types 0-8, Section Extensions 1-11, symbol scheduling, beam id, numerology.

Field	Width	Legal	Cover
version	4	const 0x1	—
section_type	4	[0:8]	enum
section_ext	4	[1:11]	enum
start_symbol	4	[0:13]	—
num_symbol	4	[1:14]	bucket4
num_sections	4	{1,2,4,8}	enum
beam_id	16	[0:65535]	—
numerology	4	[0:4]	enum
seq	8	[0:255]	wrap

Cross: $\text{start_symbol} + \text{num_symbol} \leq 14$ (slot symbol ceiling). Assertions: symbol scheduling bound; section type/extension legal; C-plane precedes U-plane by T12 (Lane-2 behavioural).

3.5 splane — S-Plane PTP + SyncE (both modes)

Purpose: synchronization — IEEE 1588v2 PTP message types and clock states, SyncE quality level, timing error budget, holdover.

Field	Width	Legal
version	4	const 0x1
ptp_msg	4	{0..5} Sync/Follow_Up/Delay_Req/Delay_Resp/Announc
clock_state	2	{0..3} MASTER/SLAVE/PASSIVE/LISTENING
sync_ql	4	{0,2,4,11,15}
seq	16	[0:65535]

timing_err	12	[0:3000] ns ($\pm 1.5\mu\text{s}$ mapped)
holdover	1	{0,1}

Assertions: Sync followed by Follow_Up; Delay_Req answered by Delay_Resp; timing error within $\pm 1.5\mu\text{s}$; SyncE SSM code valid.

3.6 mplane — M-Plane NETCONF/YANG (both modes)

Purpose: management plane — NETCONF operations, datastore selection, RU config item, antenna calibration, software slot.

Field	Width	Legal	Cover
version	4	const 0x1	—
netconf_op	3	{0..4} get/get-config/edit-config/rpc/notification	enum
datastore	2	{0,1,2} running/candidate/startup	enum
item_id	16	[0:65535]	—
ant_cal	1	{0,1}	bool
sw_slot	3	[0:7]	—
seq	8	[0:255]	wrap

3.7 beamforming — Beamforming Engine (5G NR)

Purpose: beam weights and codebook — Section Extensions 1/4/5/6, up to 64 antenna ports, per-layer weights.

Field	Width	Legal	Cover
version	4	const 0x1	—
beam_id	16	[0:65535]	—
num_ports	8	{1,2,4,8,16,32,64}	enum
num_layers	4	[1:8]	—
section_ext	4	{1,4,5,6}	enum
codebook_idx	16	[0:65535]	—
seq	8	[0:255]	wrap

Cross: $\text{num_layers} \leq \text{num_ports}$. Assertions: beam id within codebook; weight array length = $\text{ports} \times \text{layers}$ (Lane-2 behavioural); extension present-flag consistency.

3.8 compression — IQ Compression (5G NR)

Purpose: IQ compression engines — BFP, μ -law, static; bit-width reduction 16→9.

Field	Width	Legal	Cover
version	4	const 0x1	—
method	2	{0,1,2} BFP/ μ -law/static	enum
iq_width	5	const 16	—
comp_width	5	{9..16}	enum
exponent	4	[0:15]	—
block_size	8	[1:255]	—
seq	8	[0:255]	wrap

Cross: $\text{comp_width} \leq \text{iq_width}$. Assertions: compress/decompress reversibility (Lane-2 behavioural, SNR > 40 dB); compressed bit-width within legal set.

3.9 prach — PRACH Handler (5G NR)

Purpose: random-access — Section Type 3, ZC sequence, formats 0/A1-A3/B1-B4/C0/C2, FR1/FR2, numerology.

Field	Width	Legal	Cover
version	4	const 0x1	—
section_type	4	const 3	—
format	4	{0..9} = 0/A1/A2/A3/B1/B2/B3/B4/C0/C2	enum
numerology	4	[0:4]	enum
fr	1	{0,1} FR1/FR2	bool
zc_root	8	[0:255]	—
occasion	6	[0:63]	—
seq	8	[0:255]	wrap

Assertions: section_type == 3; format in legal set; time offset within format bounds (Lane-2 behavioural).

3.10 mimo_massive — Massive MIMO (5G NR)

Purpose: up to 64T64R spatial multiplexing — antenna config, layers, TDD config, rank, precoder.

Field	Width	Legal	Cover
version	4	const 0x1	—
ant_cfg	8	{1,2,4,8,16,32,64}	enum
num_layers	4	[1:8]	—
tdd_cfg	3	[0:6]	enum
rank	4	[1:8]	—
precoder_idx	8	[0:255]	—
seq	8	[0:255]	wrap

Cross: num_layers ≤ ant_cfg and rank ≤ num_layers. Assertions: layer count ≤ ports; no UL IQ in DL-only slot (Lane-2 behavioural).

3.11 bwp — BWP Manager (5G NR)

Purpose: dynamic bandwidth-part switching — up to 4 BWPs, numerology per BWP, FR1/FR2 bandwidths.

Field	Width	Legal	Cov
version	4	const 0x1	—
bwp_id	2	[0:3]	enu
numerology	4	[0:4]	enu
bw_mhz	9	{5,10,15,20,25,40,50,60,80,100,200,400}	enu
fr	1	{0,1}	boo
active	1	{0,1}	boo
seq	8	[0:255]	wra

Assertions: active BWP id in [0:3]; numerology change at slot boundary; FR2 ⇒ μ ≥ 2 (Lane-2 behavioural).

3.12 mmwave — mmWave FR2 Handler (5G NR)

Purpose: FR2 operation — bands n257/n258/n260/n261, numerology μ=2/3/4, SCS 60/120/240, SSB periodicity, beam management.

Field	Width	Legal	Cover
-------	-------	-------	-------

version	4	const 0x1	—
band	3	{0,1,2,3} n257/n258/n260/n261	enum
numerology	4	[2:4]	enum
scs	2	{0,1,2} 60/120/240 kHz	enum
ssb_period	4	{0..5} 5/10/20/40/80/160 ms	enum
beam_id	16	[0:65535]	—
seq	8	[0:255]	wrap

Assertions: numerology ≥ 2 for FR2; band/SCS consistency.

3.13 laa — LAA Handler (5G NR)

Purpose: Licensed Assisted Access — Section Type 5, listen-before-talk categories and results, burst type, channel access priority, Section Extension 3.

Field	Width	Legal	Cover
version	4	const 0x1	—
section_type	4	const 5	—
lbt_cat	3	{1,2,3,4}	enum
lbt_result	1	{0,1} SUCCESS/FAILURE	bool
burst_type	1	{0,1} partial/full	bool
cap	2	[0:3] channel access priority	enum
section_ext	4	const 3	—
seq	8	[0:255]	wrap

Assertions: section_type == 5; LBT SUCCESS before burst data; no U-plane data on LBT FAILURE (Lane-2 behavioural); Section Extension 3 present with Type 5.

4. Protocol Constants Reference

Item	Values
Numerology μ	0 (15 kHz) / 1 (30 kHz) / 2 (60 kHz) / 3 (120 kHz) / 4 (240 kHz)
Symbols per slot	14
PRB ceilings	FR1 ≤ 275 , FR2 ≤ 66
eCPRI message types	0 IQ / 1 Bit / 2 RT-control / 3 Generic / 4 RMA / 5 Delay / 6 Reset / 7 Event
C-plane Section Types	0-8
Section Extensions	1-11
FR2 bands	n257 (28 GHz) / n258 (26 GHz) / n260 (39 GHz) / n261 (28 GHz)
Compression	uncompressed / BFP / μ -law / static; 9-16 bit

4a. RAL — M-Plane RU Configuration Register Model (uvm_reg)

The suite includes a native-UVM register abstraction layer modelling the O-RAN M-plane Radio-Unit configuration register file (lane2/ral/oran_ral_pkg.sv). It is the standard uvm_reg architecture: a register block with a little-endian map, typed fields with access policy, an adapter translating abstract uvm_reg_bus_op to the concrete bus item, and a driver modelling the register memory (a stand-in DUT). Field-level design:

Register	Offset	Access	Field	Bits	Reset	Semantic
ru_ctrl_reg	0x00	RW	enable	[0]	0	RU enable
		RW	mode	[2:1]	0	operation mode
		RW	antmap	[10:3]	0	antenna map (read-protect)
ru_version_reg	0x04	RO	major	[7:0]	0x02	HW version (read-protect)
		RO	minor	[15:8]	0x0A	HW version (read-protect)
ru_numant_reg	0x08	RW	n_tx	[6:0]	1	# Tx antennas
		RW	n_rx	[14:8]	1	# Rx antennas
ru_comp_reg	0x0C	RW	method	[1:0]	1	IQ compensation method
		RW	width	[6:2]	16	IQ sampling width
ru_bwp_reg	0x10	RW	bwp_id	[1:0]	0	bandwidth part ID
		RW	numerology	[5:2]	0	μ (0..4)
		RW	active	[6]	0	BWP active flag

Verification sequence (oran_ra1_seq): (1) reset() then confirm the mirror equals the programmed reset value; (2) read the RO version register and check it returns 0x0A02; (3) for each RW register, 20 iterations of randomize() → update() (front-door write via adapter) → read() → compare read data against the desired mirror; (4) attempt a write to the RO version register and confirm the value is unchanged (write-protect). Access uses the default map with set_auto_predict(1). Executed: 163 register operations, 0 mismatches.

5. Out of Scope (behavioural — Lane-2 / future)

The suite models components at the header/transaction level. Deep behavioural layers are documented extensions on the same lane structure and are not part of the current signoff: T12/T34 fronthaul timing servo; beam-weight math (SVD, codebook precoding); compression SNR fidelity; PTP servo and holdover dynamics; concatenation reassembly across Ethernet frames; per-slot per-symbol ordering scoreboard.